

Job Shop Scheduling Problem: A Genetic Algorithm Approach

Yusuf Salad Yusuf · yuyus9282 · Joanne Teves Farstad

ACIT4610

1. Introduction

The Job Shop Scheduling Problem (JSSP) requires sequencing a set of jobs, each consisting of an ordered series of operations, across a set of machines, such that every operation runs on its designated machine for a fixed duration, machines process one operation at a time, and each job's operations respect their required order. The objective addressed here is minimisation of the makespan (C), the completion $\max$ time of the last operation across all jobs. This report develops and evaluates a Genetic Algorithm (GA), implemented from scratch in Python, to solve the JSSP using benchmark instances from the Lawrence ('la') family within the public JSPLib library. Two instances were selected per problem-size category: la01 and la02 (10 jobs, 5 machines, "small"), la16 and la17 (10 jobs, 10 machines, "medium"), and la31 and la32 (30 jobs, 10 machines, "large"), allowing performance to be compared across increasing problem scale.

2. Chromosome Representation

The chromosome uses an operation-based encoding: a flat list of job identifiers whose length equals the instance's total number of operations. Each job j appears in the list exactly as many times as it has operations — for example, in a 10-job, 5-machine instance, every job ID appears five times across the 50-gene chromosome. Reading the sequence left to right, the k -th occurrence of job j is always interpreted as job j 's k -th operation, in the fixed order that job's operations must be processed. This interpretation is what allows the chromosome to remain implicitly valid under any reordering: because validity depends only on how many times each job label appears, not on where those labels sit, shuffling the gene order can never violate a job's internal precedence constraint. Consequently, every possible permutation of the correct job-count multiset represents a feasible operation sequence, and no repair mechanism is required after initialization, crossover, or mutation.

Position i	Gene (Job)	Occurrence count k	Operation number of Job	Machine, Duration
0	1	1 st	1 o of first j1	(0, 21)
1	0	1 st	1 o of second j0	(1, 21)
2	0	2 nd	2 o of j1	(0, 53)
3	1	2 nd	2 o of j0	(3, 52)
4	0	3 rd	3 o of j0	(4, 95)
5	1	3 rd	3 o of j1	(4, 16)

Table 1: Decoding a shuffled chromosome combining Job 0 and Job 1's operations from la01. Gene value 1 appears at positions 0 and 3, and gene value 0 appears at positions 1, 2 and 4, yet each occurrence still resolves to the correct, sequential operation for its job — position and occurrence count diverge almost immediately (Job 0's second

operation sits at position 2, not position 1), confirming that decoding depends only on occurrence count per job, never on array position.

3. Schedule Building Algorithm (SBA) Design

Chromosomes are decoded into schedules using a semi-active Schedule Building Algorithm. The decoder walks the chromosome once, maintaining two arrays: `job_ready`, recording when each job is free to begin its next operation, and `machine_free`, recording when each machine next becomes available. For each gene, the corresponding operation is scheduled to start at $\max(\text{job_ready}[\text{job}], \text{machine_free}[\text{machine}])$ and finish after its processing duration — a single rule that simultaneously enforces both JSSP constraints: that a job's operations run in their required order, and that a machine processes only one operation at a time. The makespan is the latest finish time across all scheduled operations. This semi-active builder always appends an operation after a machine's currently-last-scheduled operation, without searching for earlier idle gaps on that machine. Active and Giffler-Thompson builders, which backfill such gaps and can produce shorter makespans, were considered during design but not implemented, given the project's time constraints — acknowledged here as a limitation rather than an oversight.

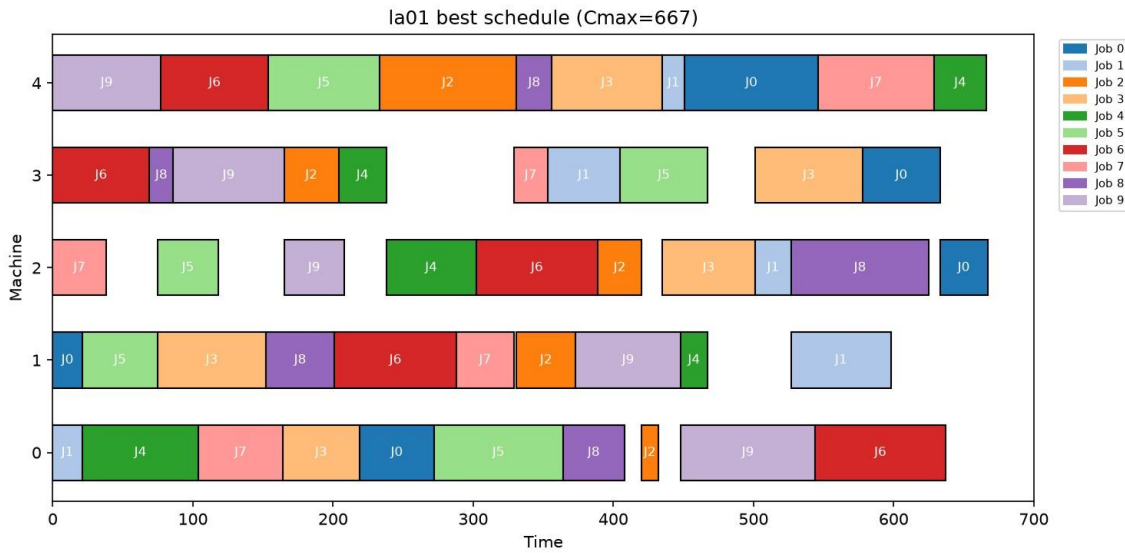


Figure 1: Best schedule found for la01 ($C = 667$, single seeded run). Each row is a machine; bars show the job $\max$ occupying it over time. No bars overlap within a machine row, confirming the decoded schedule is valid and non-overlapping.

4. Genetic Algorithm Operators

Parent selection uses k-way tournament selection: k individuals are sampled at random from the population, and the one with the lowest makespan among them is chosen, repeated to fill a mating pool. Because the operation-based chromosome allows repeated job labels, the standard Order Crossover used for permutation problems like the TSP cannot be applied directly — that version assumes every value appears exactly once, and copying a contiguous positional segment could easily copy too many or too few occurrences of a given job, breaking the chromosome's validity. The adapted operator instead partitions by job label rather than by position: a random subset of job IDs is chosen, every occurrence of those jobs is copied from parent 1 at its original positions, and the remaining positions are filled by scanning parent 2, in order, for occurrences of the other job IDs. This guarantees the correct job-count multiset regardless of how

it is distributed across positions. Mutation applies inversion — reversing a randomly chosen sub-segment, which only reorders existing genes and can never alter how many times each job appears.

5. GA Parameters

Three parameter configurations were tested to examine the trade-off between solution quality and computational cost (Table 2). All three configurations applied 3-way tournament selection and elitism, retaining the single best individual unchanged each generation. Each configuration was run independently 10 times per instance, using a distinct random seed per run to account for the GA's inherent stochasticity, and the resulting makespans were aggregated into best, worst, mean, standard deviation, execution time, and convergence generation.

Parameter Set	Population	Generations	Crossover Prob.	Mutation Prob.
A (Light)	50	100	0.70	0.05
B (Medium)	100	200	0.80	0.10
C (Heavy)	150	300	0.90	0.15

Table 2: The three GA parameter configurations evaluated.

6. Results and Comparison

Across all six instances, the GA consistently produced solutions close to the published JSPLib optima, with the gap narrowing as parameter intensity increased. Under the lightest configuration (Set A), the average gap to the known optimum across all instances was 9.57%; Set B reduced this to 4.30%; and Set C achieved the closest results overall, averaging a 2.03% gap. On the small instances, Set C reached the exact known optimum for la01 (666) and came within 0.76% of optimal for la02 (660 versus 655). On the large instances, even Set C remained further from optimal (1.01% for la31, 5.35% for la32), reflecting the substantially larger search space of a 30-job, 10-machine problem, whose chromosome length (150 operations) is three times that of the small instances.

Comparing the three problem-size categories directly (Table 3) shows the same pattern: the small category averaged a 1.81% optimality gap, 1.29 seconds of execution time, and convergence at generation 67.6; the medium category averaged 4.57%, 2.39 seconds, and generation 92.5; the large category averaged 9.52%, 6.38 seconds, and generation 140.2. Both execution time and the convergence generation increased substantially with problem size, consistent with a larger search space requiring more generations — and more computation per generation — to explore.

Category	Mean Gap to Optimum	Mean Exec. Time (s)	Mean Convergence Gen.
Small (10×5)	1.81%	1.29	67.6
Medium (10×10)	4.57%	2.39	92.5
Large (30×10)	9.52%	6.38	140.2

Table 3: Results averaged by problem-size category (across both instances and all 3 parameter sets).

The three parameter sets (Table 4) showed a clear quality-versus-cost trade-off. Increasing population size and generation count reduced the average optimality gap by more than three-quarters (9.57% to 2.03%)

while increasing average execution time nearly ninefold (0.71s to 6.49s) and delaying convergence from generation 54.7 to 134.9. Standard deviation also fell substantially with heavier settings (41.4 to 23.0 on average), indicating that larger populations produced more consistent results across runs, not merely better best-case outcomes.

Parameter Set	Mean Gap to Optimum	Mean Exec. Time (s)	Mean Convergence Gen.	Mean Std. Dev.
A (Light)	9.57%	0.71	54.7	41.4
B (Medium)	4.30%	2.86	110.6	33.2
C (Heavy)	2.03%	6.49	134.9	23.0

Table 4: Results averaged by GA parameter set (across all six instances).

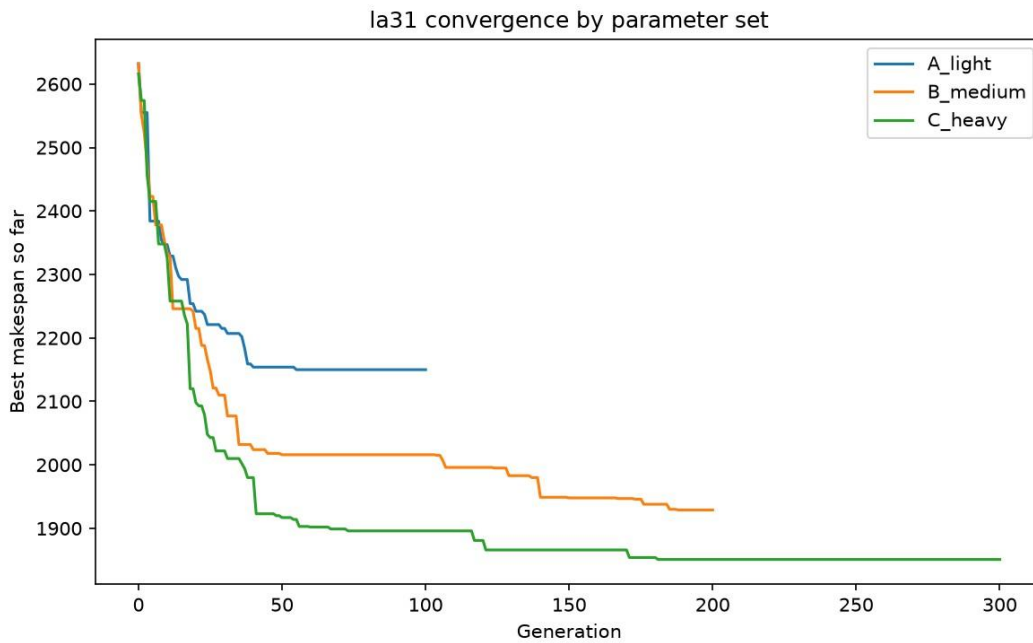


Figure 2: Convergence of best-found makespan by generation for la31, one representative run per parameter set. All three curves are strictly non-increasing (a consequence of elitism) and flatten once no further improvement is found; Set C reaches the lowest value but takes the most generations to do so, matching the quality/cost trade-off in Table 4.

One result deviates from this trend and is reported rather than concealed: for la16, Set C's best makespan (982) was marginally worse than Set B's best (975), the only instance where a heavier configuration did not yield the best single result. However, Set C's mean (994.9) and standard deviation (13.8) both improved on Set B's (1025.8 and 31.7), indicating Set C remained the more reliable configuration overall; the discrepancy reflects the GA's inherent stochasticity rather than a design flaw, and is precisely why mean and standard deviation, not a single best value, are reported as primary comparison metrics.

7. Discussion and Limitations

Two design decisions, made under the project's time constraints, are worth stating explicitly rather than leaving implicit. First, the SBA decoder implemented is semi-active: it never backfills idle gaps earlier on a machine's timeline, unlike an active or Giffler-Thompson builder. This likely explains part of the remaining optimality gap, particularly on the larger instances, and represents the most promising direction for improving solution quality in future work. Second, each configuration was run 10 times per instance rather than the

upper end of the assignment's permitted 10–30 range; while sufficient to compute stable statistics, additional runs would tighten the reported standard deviations further, especially for the large-instance, high-variance configurations. Both choices were made deliberately to fit the available time, and are stated here as scoping decisions rather than oversights.

8. Conclusion

This project implemented a Genetic Algorithm using an operation-based chromosome, an adapted Order Crossover, inversion mutation, and a semi-active Schedule Building Algorithm, to solve the Job Shop Scheduling Problem across six JSPLib benchmark instances spanning three problem sizes. My results show the GA consistently approaches known-optimal makespans, with the gap widening predictably as problem size increases, and narrowing substantially as population size and generation count increase, at a proportional cost in execution time. My findings confirm both our expected output of the implementation and the expected trade-off between solution quality and computational effort inherent to evolutionary search.

References

JSPLib benchmark library (Lawrence, 1984 'la' instance set): <https://github.com/tamy0612/JSPLIB>